JOURNAL OF PARALLEL AND DISTRIBUTED COMPUTING **34,** 36–49 (1996)
ARTICLE NO. 0044

Matrix Transpose for Block Allocations on Torus and de Bruijn Networks

CHRISTOPHE CALVIN AND DENIS TRYSTRAM

LMC-IMAG, Institut Fourier, B.P. 53X, 100 rue des Mathématiques, 38041 Grenoble Cedex 9, France

We present in this paper fast algorithms for the matrix transpose problem on distributed-memory parallel machines for block allocations of the matrix. We propose a general methodology based on a partitioning of the communication networks into paths or cycles for which we develop optimal algorithms for elementary pairwise exchanges. It leads to efficient algorithms for torus and de Bruijn networks. We study the lower bounds for the linear model of time for both store-and-forward and wormhole commutation modes, and we compare the execution time of our algorithms with these lower bounds. © 1996 Academic Press, Inc.

1. INTRODUCTION

Most parallel numerical methods need regular data movements [8, 15, 27]. In this context, the matrix transpose is one of the most useful collective communication routines (FFT, matrix–vector, and matrix–matrix product) [6, 12, 13, 17].

We study in this paper the implementation of the matrix transpose on distributed-memory parallel computers for block allocations. We propose a general methodology based on decompositions of the communication networks into paths or cycles. It leads to obtain easily efficient algorithms for networks which admit such a decomposition. We apply this framework on torus and de Bruijn networks.

1.1. Model

Let us consider P processors connected by a communication network. We assume either *store-and-forward* (denoted S&F) or *wormhole* commutation mode [17, 25]. As it is the case in most existing parallel machines, we assume that the communication links are *full-duplex* (i.e., they can be used simultaneously in both directions). We will assume also a k-port model (with $k = 2, 4$), i.e., a processor can use k links at the same time. A justification of this model can be found in [22, 25].

The communication time between neighbor processors is linear: the cost for sending a message of size L can be expressed as

$$T_{comm} = \beta + L\tau,$$

where L is the length of the message, β the start-up time, and τ the transmission rate.

The time for sending a message from one node to another located at distance d is modeled as follows:

- $T_{comm}^{S\&F} = d(\beta + L\tau)$ using S&F
- $T_{comm}^{Worm} = \beta + d\delta + L\tau$ using wormhole.

In most parallel machines the influence of the distance is negligible if the communication is contention free [22, 25]. So we will model a communication of a message of size L between two distant processors by $T_{comm}^{Worm} = \beta + L\tau$.

1.2. Organization of the Paper

We recall in Section 2 the definitions of the target networks: hypercube, torus, and de Bruijn. We describe the cyclic block matrix allocation and we define the matrix transpose problem. Section 3 is devoted to a review of previous works using S&F commutation mode: we recall first the principle of the recursive matrix transpose algorithm and its implementation on hypercube; then we recall some existing results on the grid and we derive some algorithms using general permutation results. Section 4 concerns preliminary results on two elementary data transposes on paths and cycles. We propose general algorithms for the matrix transpose on any network partitioned into such paths or cycles for S&F commutation mode. This methodology is applied to torus and de Bruijn networks in Section 5. We improve these algorithms by using reverse idle links and derive complexity results. We study then the lower bounds using S&F and verify that the algorithm proposed for the de Bruijn is optimal for the start-up factor and asymptotically optimal for the transmission time (when the matrix size or the number of processor increases) and that the algorithm for the torus is asymptotically optimal for start-up time and is about a factor 2 of the transmission rate lower bound (when the matrix size increases). Before concluding we describe the adaptation of these preceding results using wormhole routing, we analyze some related works on the same problem on 2D-meshes, and we determine the lower bounds of the matrix transpose problem with wormhole commutation mode.

0743-7315/96 $18.00
Copyright © 1996 by Academic Press, Inc.
All rights of reproduction in any form reserved.

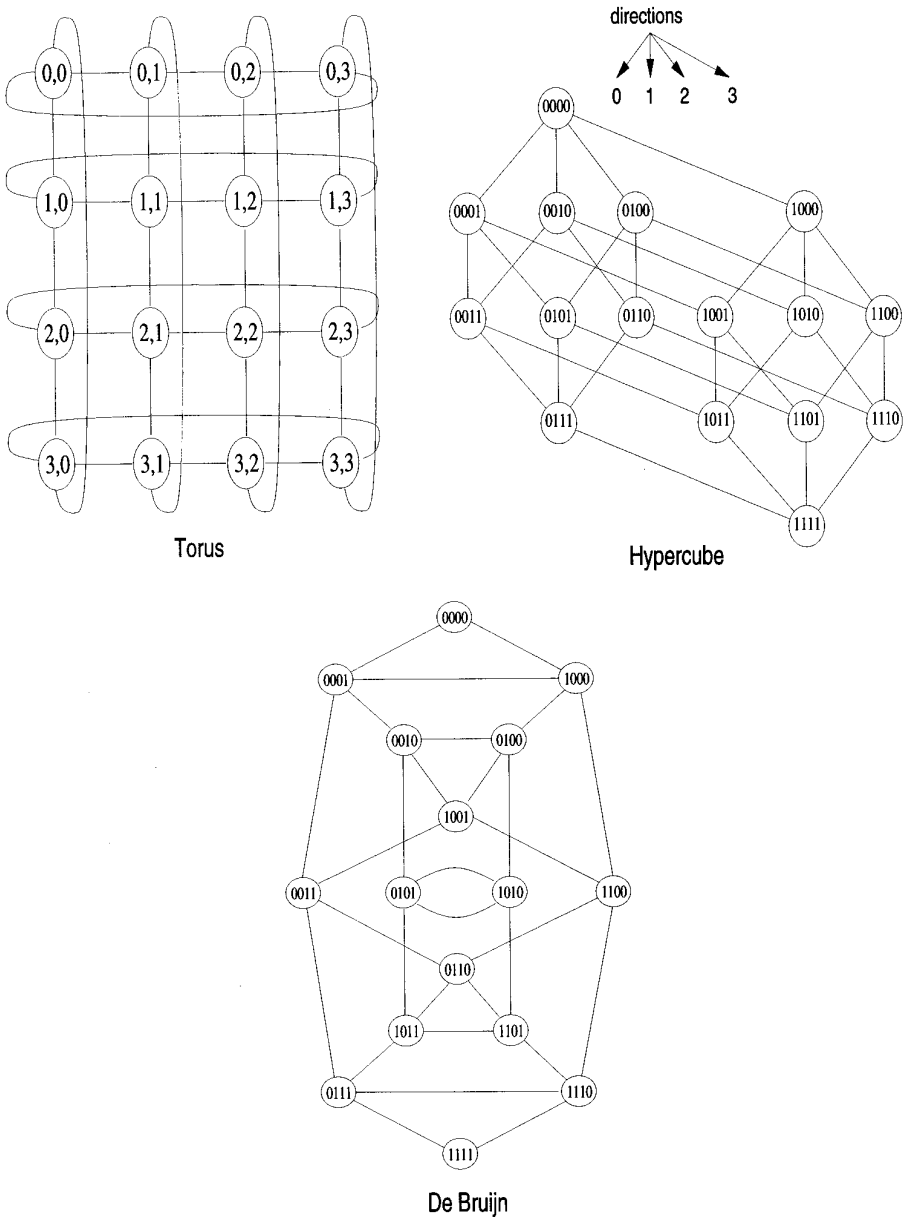


FIG. 1. Target networks and node numbering ($P = 16$).

2. BRIEF DESCRIPTION OF NETWORKS AND DATA ALLOCATION

The networks considered in this paper are hypercube, grid, torus, and de Bruijn (see Fig. 1). We assume that the processors are numbered from 0 to $P - 1$. We briefly recall the definitions of these networks [14, 19, 25].

2.1. The Communication Networks

Hypercube. The hypercube of dimension n has $P = 2^n$ nodes. The nodes are represented by all the binary words of length n. Two nodes are linked along direction k, if their representation differ only from bit k for $0 \leq k \leq n - 1$. So, a node has exactly n neighbors [26].

Grid and torus. We consider square grids and torus of size $\sqrt{P} \times \sqrt{P}$ where $\sqrt{P}$ is an integer.[1] A node is an ordered pair (q_1, q_2) of integers, for $0 \leq q_1, q_2 \leq \sqrt{P} - 1$. The torus is a well-known network whose edges connect nodes which differ by 1 mod $\sqrt{P}$ in exactly one coordinate. Each node has four neighbors. In grids, nodes which are on the boundaries ($q_1 = 0$, $q_1 = \sqrt{P} - 1$, $q_2 = 0$, $q_2 = \sqrt{P} - 1$) are only linked with internal nodes.

De Bruijn. The de Bruijn network is composed of nodes which can be represented by binary words of length

[1] The justification of this hypothesis can be found in the next subsection.

n. There is an arc from node $(i_0 \cdots i_{n-1})$ to nodes $(i_1 \cdots i_{n-1}0)$ and $(i_1 \cdots i_{n-1}1)$ [1, 10].

The construction is oriented, but we forget the orientation of the arcs and remove simple loops.

2.2. Data Allocation

Given an $N \times N$ matrix A and P processors, there are many ways to allocate the entries of a matrix [8, 16, 23, 24]. For the sake of simplicity, we assume in the following that N/P is an integer.

• *Row* (*column*) *wise allocation.* The matrix can be divided into P load-balanced blocks of size N/P rows (or columns) [23]. For this kind of allocation, the matrix transpose corresponds to a personalized total exchange on the network [17]: each processor must send different parts of its rows (or columns) to each processor, that is,

$$\frac{1}{P} \times \left(N \times \frac{N}{P} \right) = \left(\frac{N}{P} \right)^2$$

data. We recall that the costs of the best personalized total exchange routines using an S&F routing mechanism with messages of size L are respectively for the torus [27] and the hypercube [6, 17]

$$\sqrt{P}\left(\beta + \frac{PL}{8}\tau\right) \quad \text{and} \quad \log_2(P)\left(\beta + \frac{L}{2\log_2(P)}\tau\right).$$

• *Cyclic block allocation functions.* Another way to allocate the matrix on the network is to subdivide it into P subblocks and to associate each subblock to a processor [3, 4]. In the general case, the matrix transpose corresponds as before to a personalized all-to-all. But if we suppose that P is such that $\sqrt{P}$ is an integer, the subblocks are square and the matrix transpose corresponds to simple exchanges between pair of processors. We will assume in the rest of the paper that $\sqrt{P}$ is an integer.[2] We present in Fig. 2 the *consecutive block allocation* (in terms of High Performance Fortran [9], the matrix is decomposed in a (*block, block*) manner) and the *cyclic block allocation* functions.

Let $(i)_2$ be the binary representation of integer i and $|$ the operator of bit concatenation. We denote $Alloc(i, j)$ the processor which contains the block (i, j) for $0 \le i, j \le (N/\sqrt{P}) - 1$. The consecutive block allocation function is defined as

$$Alloc(i,j) = \left(\left\lfloor i \times \frac{N}{\sqrt{P}} \right\rfloor \right)_2 \Big| \left(\left\lfloor j \times \frac{N}{\sqrt{P}} \right\rfloor \right)_2.$$

[2] This implies that the dimension of both hypercube and de Bruijn networks, i.e., n, is even.

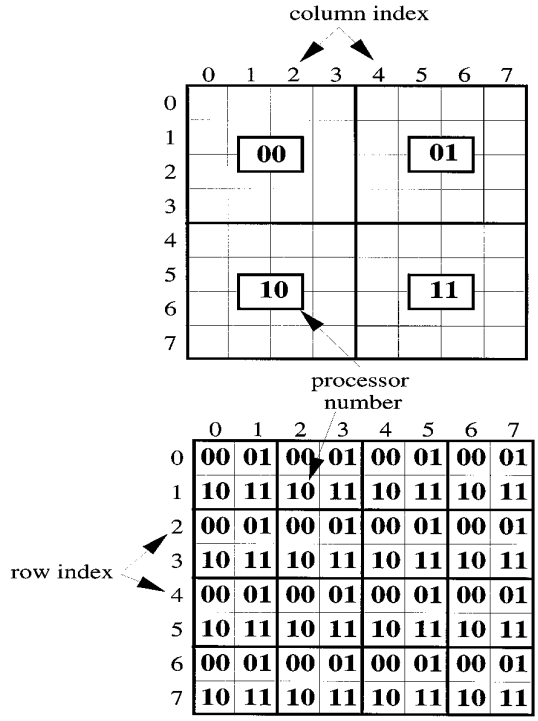


FIG. 2. Consecutive and cyclic block allocations ($N = 8$, $P = 4$).

The algorithms that we propose in this paper for transposing a matrix are available for any block allocation functions. For the sake of simplicity, we will only consider the consecutive block allocation function. The results for other standard block allocations are the same.

There exist many other allocation functions, especially some which allow the matrix transpose problem to be reduced to neighbor communications [13, 20]. However we consider here only standard block allocation functions [4, 9].

3. REVIEW OF PREVIOUS WORKS USING STORE-AND-FORWARD

3.1. Recursive Matrix Transpose

The matrix transpose is a well-known operation which associates the element in row i and column j with that in row j and column i. Using block allocation function, a processor needs to exchange a given block with only one processor. More precisely, processor $Alloc(i, j)$ exchanges its block with processor $Alloc(j, i)$. We call this operation a *direct transpose.*

Using this block decomposition, the matrix transpose can be realized recursively in $\log_2(N)$ steps given in the algorithm in Fig. 3 [7]:

$$A = \begin{bmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{bmatrix}.$$

```
Procedure Transpose(A)
    Exchange sub-blocks A01 and A10
    Transpose(A00)
    Transpose(A01)
    Transpose(A10)
    Transpose(A11)
```

FIG. 3. Recursive transpose algorithm.

3.2. Implementation on Hypercube

The recursive construction of the hypercube allows a direct implementation of the previous algorithm [17].

The binary representation of element (i, j) of matrix A is $(i)_2|(j)_2$. Step s of the algorithm consists in exchanging bits s and $(s + (\log_2(N)/2))$ of the binary representation of element (i, j) of the matrix, for $s = 0$ to $(\log_2(N)/2) - 1$ (see Fig. 4).

The first $\log_2(P)/2$ steps involve communications between processors. It is clear that each step can be realized in two basic communication steps (see Fig. 5) with data of size N^2/P, so the total communication time of the algorithm is

$$T_1^{Hyp-\mathrm{S\&F}} = \log_2(P)\left(\beta + \frac{N^2}{P}\tau\right).$$

If we use bidirectional links, the blocks can be divided into two equal parts. Each message is sent in both directions at each step. The cost of the algorithm becomes [16]

$$T_2^{Hyp-\mathrm{S\&F}} = \log_2(P)\left(\beta + \frac{N^2}{2P}\tau\right) \quad [18].$$

Let us remark that the factor of transmission rate (τ) is proportional to half the diameter. We will see that we obtain better results with our method applied to torus and de Bruijn.

Johnsson and Ho [16] described the *multiple paths recursive transpose* algorithm which uses different paths between source nodes and destination nodes. Using a pipelining technique, it leads to a very efficient algorithm, since its cost is (for large N)

$$T_3^{Hyp-\mathrm{S\&F}} = \left(\sqrt{\beta} + \sqrt{\frac{N^2}{2P}\tau}\right)^2.$$

Let us remark that asymptotically they obtain the best transmission rate as possible.

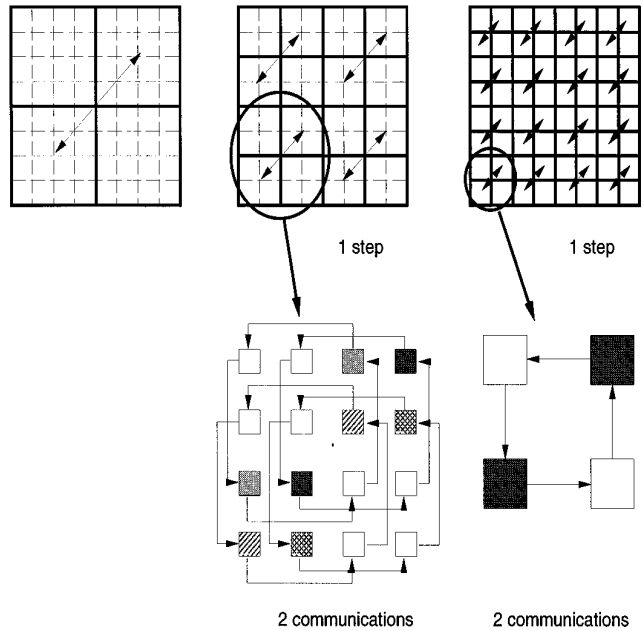


FIG. 5. Implementation of recursive matrix transpose on hypercube.

3.3. Previous Work on the Grid

In [15] Johnsson described a matrix transpose algorithm with $2^k \times 2^k$ grids in 2^k steps of direct neighbor communications, by shifting successively superdiagonals in the direction of decreasing column indices and then in the direction of increasing row indices. Symmetrically, subdiagonals are shifted first in the direction of decreasing row indices and then in the direction of increasing column indices (see Fig. 6 for the successive steps). Johnsson considered a restricted communication model with no start-up time. It is clear that the cost of step i is equal to $2i$, which leads to a total time of $2(2^k - 1)$. He proposed the idea of an extension of this algorithm for torus, which uses distinct paths and pipelining but without detailing it.

Using the general model of time, it leads to

$$T_1^{Grid-\mathrm{S\&F}} = 2(\sqrt{P} - 1)\left(\beta + \frac{N^2}{P}\tau\right).$$

3.4. Using General Permutation Routing

The matrix transpose problem corresponds to a permutation which belongs to the class of *linear complement permutations.* This class is defined as follows: each bit of the binary representation of the destination processor is a linear combination of the bits of the source processor [2]. It is easy to verify that the matrix transpose fits into this class.

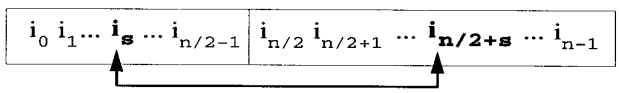


FIG. 4. Bit exchange in recursive transpose on hypercube.

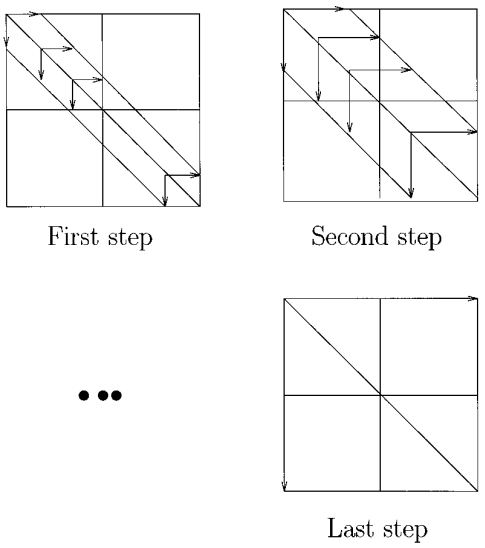


FIG. 6. Principle of the Johnsson's algorithm.

Boppana and Raghavendra propose in [2] an optimal algorithm for routing any linear complement permutation on the hypercube (in $\log_2(P)$ steps). Applying this routing algorithm to the matrix transpose problem leads to

$$T_4^{Hyp-\mathrm{S\&F}} = \log_2(P)\left(\beta + \frac{N^2}{2P}\tau\right).$$

We can derive an algorithm for matrix transpose on de Bruijn. Since this routing is done dimension after dimension, we can use this general routing algorithm for emulating the de Bruijn network from the hypercube [19]. Thus, we obtain an algorithm in $2\log_2(P)$ steps (each communication in the hypercube along one dimension can be realized in two communications in the de Bruijn), which is proportional to twice the diameter. The cost of this algorithm is equal to

$$T_1^{deBruijn-\mathrm{S\&F}} = 2\log_2(P)\left(\beta + \frac{N^2}{2P}\tau\right).$$

The same method can be applied to the torus, each step involving dimension i corresponding to processor at distance $\sqrt{P}/2^i$ in the torus. So, we obtain an algorithm whose number of steps is

$$T_1^{Torus-\mathrm{S\&F}} = \sum_{i=1}^{\log_2(P)} \frac{\sqrt{P}}{2^i} = 2\sqrt{P}\frac{P-1}{P}.$$

The specific matrix transpose algorithms on de Bruijn and torus that we propose are better than these.

4. SOME PRELIMINARY RESULTS

We present in this section two algorithms for basic data movements which are useful for designing matrix transpose algorithms on different networks.

```
PT-S&F(A,path k)
(Program of P_q on path k)
   B = local block of the matrix A
   for S = 0 to l − 1 do
      if (q = S)
         Send B to P_(q+1) on path k
      if ((q > S) and
         (q < l/2 + ⌊(S+1)/2⌋))
         Send B to P_(q+1) on path k
         Recv B from P_(q−1) on path k
      if (q = l/2 + ⌊(S+1)/2⌋)
         Recv B from P_(q−1) on path k
      if (l − q = S)
         Send B to P_(q−1) on path k
      if ((l − q > S) and
         (l − q < l/2 + ⌊(S+1)/2⌋)))
         Send B to P_(q−1) on path k
         Recv B from P_(q+1) on path k
      if (q = (l−1)/2 + ⌊(S+1)/2⌋)
         Recv B from P_(q+1) on path k
```

FIG. 7. Algorithm PT using S&F.

4.1. Path Transpose

DEFINITION. Let us consider a path of length l, with l even (where each processor is indexed from 0 to $l-1$). "Path transpose" (denoted PT) is the communication scheme where each processor q sends its L data to processor $l-q$, $0 \le q \le l$.

We detail the successive steps in Fig. 8 for $l = 4$. The ordered pair on each arc indicates (*initiator, addressee*). The algorithm using S&F routing mechanism is expressed in Fig. 7.

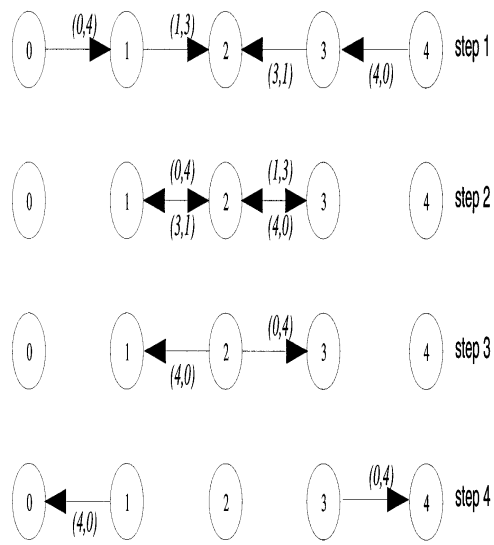


FIG. 8. PT-S&F on a path of five processors.

It is easy to verify that PT-S&F on l processors can be done in l steps; it leads to time

$$T_{\text{PT-S\&F}} = l(\beta + L\tau).$$

Improvement: Pipelined PT. During the S&F version of the algorithm, all the links are not used. In order to use the maximum number of links as possible, we divide the messages into ν packets of size L/ν and we pipeline the sendings.

Using a direct pipelining of the previous algorithm is impossible since all the intermediate processors must transmit messages. However, it is possible to pipeline by delaying periodically the sendings. We present in Fig. 9 the pipelined algorithm (denoted PPT) in more detail (for the sake of simplicity, we have only represented the sendings

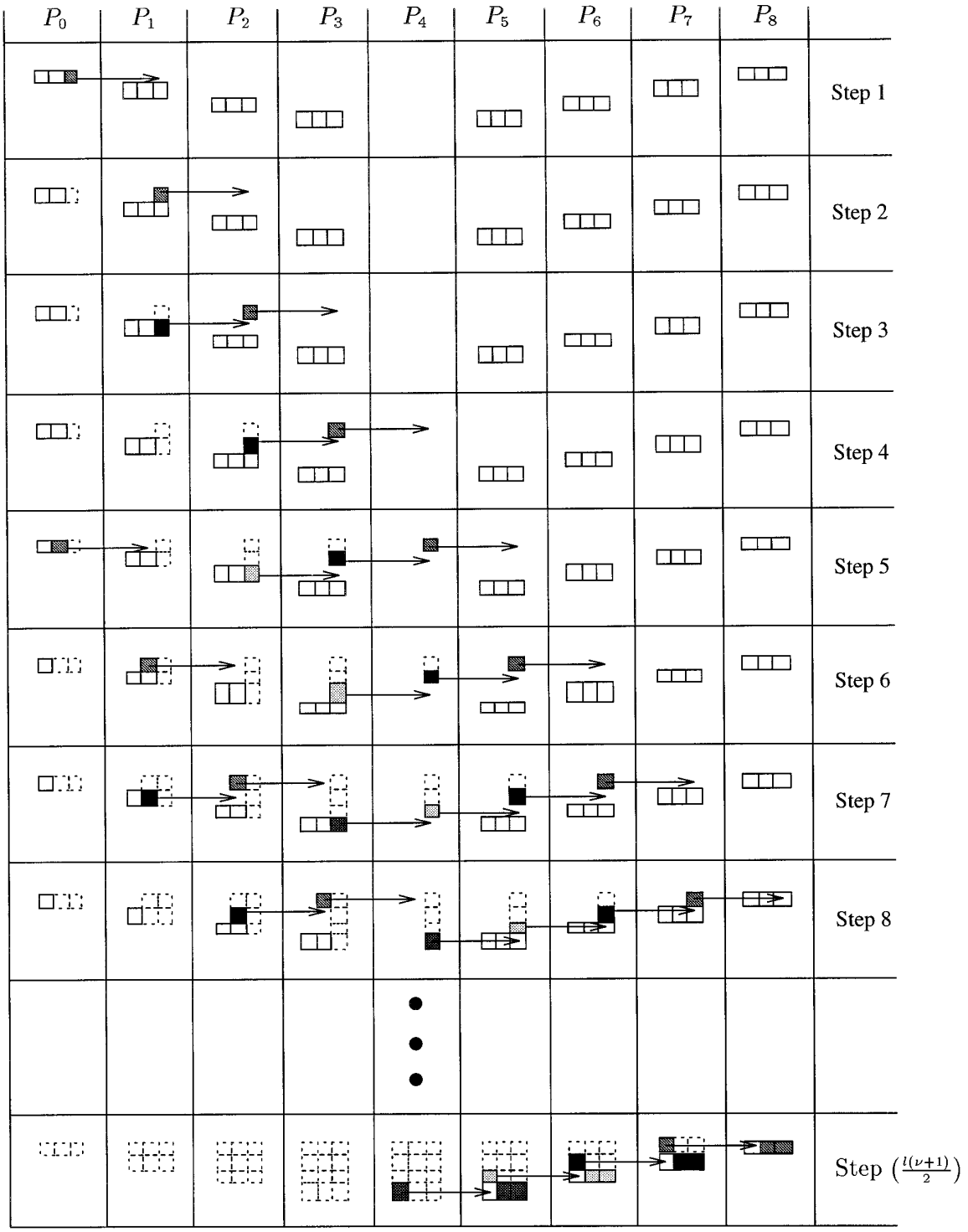


FIG. 9. PPT on a path of nine processors.

```
CT-S&F(A,cycle k)
(Program of P_q on cycle k)
   for S = 0 to ((l+1)/2) do
      Send B to P_(q+1) on cycle k
      Recv B from P_(q-1) on cycle k
```

FIG. 10. Algorithm CT using S&F.

and not the exchanges). At the first step of the algorithm, processor 0 sends its first packet. Processor 1 sends its first only when it has transmitted the packet from processor 0, i.e., at step 2. Processor 0 sends its second packet when its first packet has reached processor $(l-1)/2$, and so on. It is obvious that no conflict occurs by construction, which guarantees the correctness of the algorithm. Let us remark that the link between processors $((l-1)/2)-1$ and $(l-1)/2$ is always busy after $(l-1)/2$ steps until the penultimate step, and that all the processors finish their exchanges at the same time.

Let us now calculate the time of this algorithm. The first packet of processor 0 reaches processor l at time $l(\beta + (L/\nu)\tau)$. The ith packet of processor 0 is sent after $(l/2)(i-1)$ steps. Thus, the total time of the algorithm is

$$T_{\mathrm{PPT}}(\nu) = l\left(\beta + \frac{L}{\nu}\tau\right) + \left(\frac{l}{2}\right)(\nu - 1)\left(\beta + \frac{L}{\nu}\tau\right).$$

The optimal number of packets, ν_{opt}, is easy to calculate. We obtain $\nu_{opt} = \sqrt{L\tau/\beta}$. Thus, the time of the PPT algorithm, for the optimal number of packets, is

$$T_{\mathrm{PPT}}(\nu_{opt}) = \left(\frac{l}{2}\right)(\sqrt{\beta} + \sqrt{L\tau})^2.$$

4.2. Cyclic Transpose

DEFINITION. Let us consider a cycle of length l, with l odd. "Cyclic transpose" (denoted CT) is the communication scheme where each processor q sends its L data to processor

$$\left(q + \frac{l+1}{2}\right) \bmod l \quad \text{for } 0 \le q \le l.$$

We give in Fig. 10 the algorithm for the CT using S&F, and detail in Fig. 11 the case $l = 5$.

This transpose is performed in $(l+1)/2$ steps:

$$T_{CT\text{-S\&F}} = \frac{l+1}{2}(\beta + L\tau).$$

Let us remark that in this case we cannot improve the algorithm by pipelining, because all the links are always used during the algorithm.

4.3. A General Methodology

From the definitions of the previous elementary data movements on linear networks, we now present a general methodology for the direct transpose on any networks as follows:

1. Find first a partition of the network into C paths or cycles, such that each node and its transposed are on the same path or cycle:
   - If C is a path (defined by the list of the nodes) then it must verify the constraint $C = X_0X_1 \cdots X_l$ with l even, such that $X_i^t = X_{l-i}$ for $i = 0 \cdots l$, where X_i^t denotes the transposed node of node X_i.
   - If C is a cycle then it must verify that $C = X_0X_1 \cdots X_l$ with l odd, such that $X_i^t = X_{i+(l+1)/2}$ for $i = 0 \cdots (l - 1/2)$.

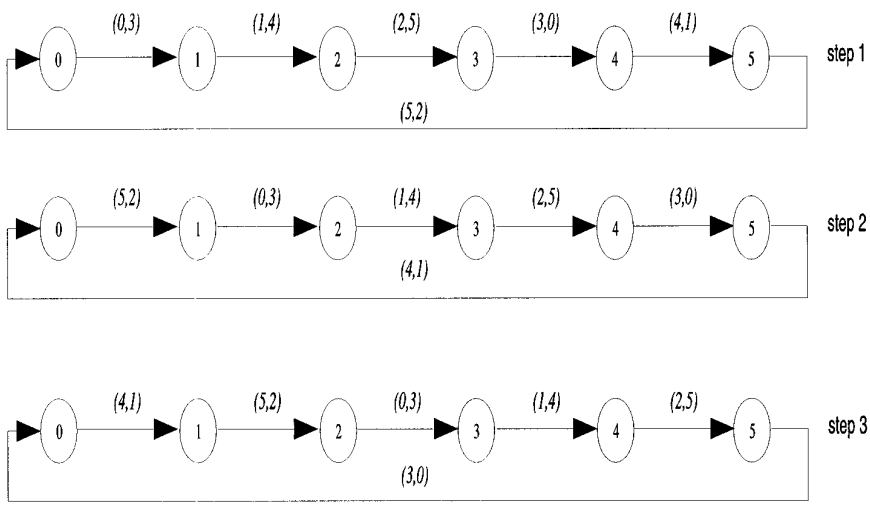


FIG. 11. CT on six processors.

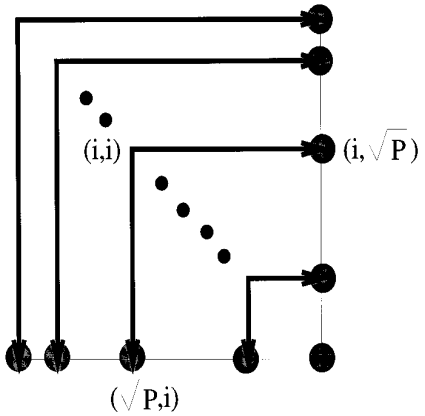


FIG. 12. Partition of the grid using PT paths.

2. Then apply the previous communication data movements on these paths or cycles.

As a consequence, the cost of a direct transpose is equal to the cost of a PPT on the longest of the C paths or a CT on the longest of the C cycles on the network.

5. APPLICATION TO USUAL NETWORKS

5.1. Grid and Torus

Matrix transpose on a grid. Let us first present a partition of a square grid of P processors. We define $\sqrt{P} - 1$ paths, numbered from $i = 0$ to $\sqrt{P} - 2$ such that path i contains node (i, i), by a list of processors along these paths. They are described in Fig. 12. These paths correspond to PT paths since they contain pairs of nodes composed by a node and its transposed node (see Section 4).

Path i contains $2(\sqrt{P} - i) + 1$ nodes, so the total number of nodes contained in all these paths is

$$\sum_{i-1}^{\sqrt{P}-1} (2(\sqrt{P} - i) + 1) = P - 1.$$

If we consider node $(\sqrt{P}, \sqrt{P})$ as a degenerated path reduced to only one processor, the set of such paths is a partition of the grid.

Thus, we obtain a direct transpose on a square grid of size $\sqrt{P}$ in $2(\sqrt{P} - 1)$ communication steps (corresponding to the longest path of the partition) involving messages of size N^2/P. So the time for the matrix transpose on a square grid using PT paths is[3]

$$T_2^{Grid-S\&F} = 2(\sqrt{P} - 1)\left(\beta + \frac{N^2}{P}\tau\right).$$

Adaptation to the torus. We can adapt the previous results to the torus using the extra wraparound links which connect the nodes of the boundaries. Figure 13 describes these PT paths.

Let us now calculate the total number of nodes contained in these paths:

- paths in A_1 (see Fig. 13):

$$\sum_{i=1}^{\sqrt{P}/2} (\sqrt{P} + 1) = \frac{1}{2}(P + \sqrt{P})$$

- paths in A_2 (see Fig. 13):

$$\sum_{i=(\sqrt{P}/2)+1}^{\sqrt{P}} (\sqrt{P} + 1) = \frac{1}{2}(P - \sqrt{P}).$$

Thus the total number of nodes contained in these paths is equal to P. All these paths are disjoint, so we obtain a partition of the torus. For the matrix transpose, we use PT paths of length equal to $\sqrt{P} - 1$ in A_1 and equal to $\sqrt{P} - 3$ in A_2.

[3] This construction is another view of the pipelined version of Johnsson's algorithm [15].

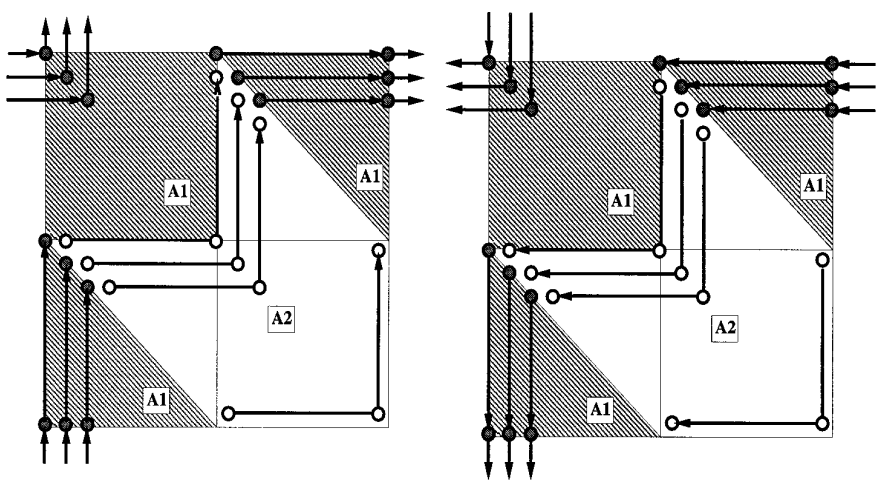


FIG. 13. Partition of the torus using PT paths.

Thus we can do a direct transpose on a torus in $\sqrt{P}$ steps of basic communication with messages of size N^2/P. So the matrix transpose time on a torus using S&F routing is

$$T_2^{Torus-\mathrm{S\&F}} = \sqrt{P}\left(\beta + \frac{N^2}{P}\tau\right).$$

We can improve this result by noting that in the previous algorithm, half of the links are idle (see Fig. 14). We split blocks into two equal parts and we use simultaneously a direct transpose on the remaining orthogonal links with messages of length $N^2/2P$. We obtain

$$T_3^{Torus-\mathrm{S\&F}} = \sqrt{P}\left(\beta + \frac{N^2}{2P}\tau\right).$$

Applying the PPT technique to the torus leads to time

$$T_4^{Torus-\mathrm{S\&F}} = \frac{\sqrt{P}}{2}\left(\sqrt{\beta} + \sqrt{\frac{N^2}{2P}\tau}\right)^2.$$

Using this technique, we obtain a time asymptotically proportional to one-fourth the diameter of the torus.

5.2. De Bruijn

The aim of this section is to find a partition of the de Bruijn into cycles as defined in Section 4.2. For this, we recall the definition of the pure cycle introduced in [10], and we prove that the decomposition of the de Bruijn into pure cycles is a partition which corresponds to that defined in Section 4.2.

A *pure cycle* is a cycle where the number of 1's in the numbering of the nodes is preserved [10]. We call *cycle generator* a node which can be used to describe a cycle by successively shifting left the n bits of the generator. For instance, if we take (1100) as a generator, the cycle is

$$[(1100)\quad (1001)\quad (0011)\quad (0110)]$$

Starting from node $(111\cdots111)$ we know how to find the list of cycle generators [10].

We give the Appendix in Table V all the cycle generators and in Table VI all the cycles for $n = 6$. It is clear that these cycles are disjoint and that a node and its transposed are on the same cycle. Now we must prove that these cycles contain all the nodes of de Bruijn.

PROPOSITION. *The total number of generators is* $N(n, 2)$ [11, 21] *with*

$$N(n,2) = \frac{1}{n}\sum_{d\in D}\phi(d)2^{n/d},$$

where D *is the set of the divisors of* n, *and* ϕ *is the Euler's totient function.*[4] *The total number of nodes contained in the cycles generated by these generators is equal to* 2^n.

For instance, for $n = 6$,

$$D = \{6, 3, 2, 1\}$$

$$\phi(6) = 2,\ \phi(3) = 2,\ \phi(2) = \phi(1) = 1$$

$$N(6,2) = \tfrac{1}{6}(\phi(6)2^{6/6} + \phi(3)2^{6/3} + \phi(2)2^{6/2} + \phi(1)2^{6/1}) = 14.$$

Proof. $N(n, 2)$ is the number of different ways of associating digits $\{0, 1\}$ in order to compose cyclic words of length n. So if we consider all these cyclic words, we enumerate all the words of length n on the alphabet $\{0, 1\}$ [21]. This is another way of describing the nodes of de Bruijn. So the $N(n, 2)$ cycles contain all the nodes of the graph and are disjoint by construction: we have a partition of the de Bruijn into $N(n, 2)$ cycles of maximal length n.

[4] $\forall n \geq 1$, $\phi(n)$ = number of integers x, $1 \leq x \leq n$, such as x and n are coprime.

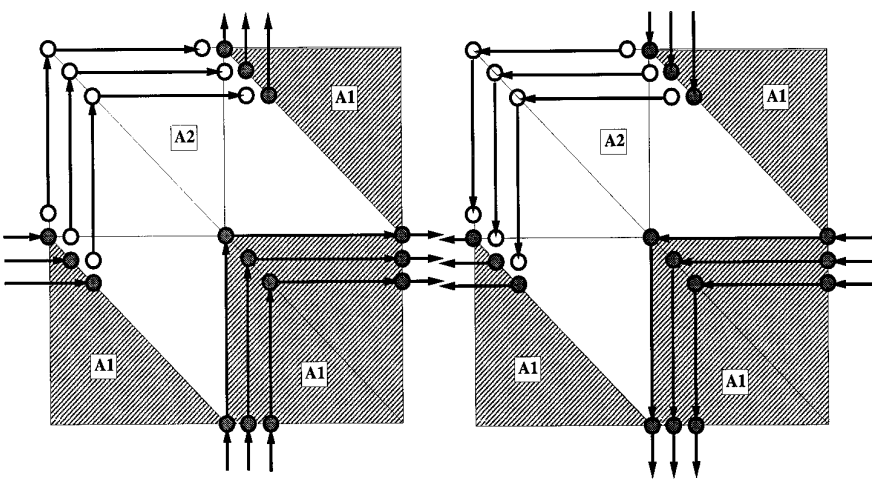


FIG. 14. The complementary partition of the torus using PT paths.

Therefore we can do a direct transpose on a de Bruijn network of size n into $n/2$ communication steps (see Fig. 15) and the transpose time is

$$T_2^{DeBruijn-S\&F} = \frac{\log_2(P)}{2}\left(\beta + \frac{N^2}{P}\tau\right).$$

This result can be improved by the same remark as previously (we use only half the links). So, if we divide messages into two parts, we can use reverse links with half the message. So the transpose time becomes

$$T_3^{DeBruijn-S\&F} = \frac{\log_2(P)}{2}\left(\beta + \frac{N^2}{2P}\tau\right).$$

Finally, we obtain for large messages a time asymptotically proportional to one-fourth the diameter of the de Bruijn, as for the torus.

6. OPTIMALITY RESULTS

6.1. Lower Bounds

We establish in this section the lower bounds for the matrix transpose on the different topologies.

PROPOSITION. *The lower bounds for the matrix transpose with consecutive or cyclic block allocations assuming store-and-forward routing mechanisms are summarized in Table I for both start-up and transmission rate factors.*

TABLE I
Lower Bound Using S&F

	Number of steps	Transmission rate
Grid	$2(\sqrt{P}-1)$	$\left\lceil\frac{\sqrt{P}}{2}\right\rceil \times \frac{N^2}{2P}$
Torus	$2\left\lfloor\frac{\sqrt{P}}{2}\right\rfloor$	$\left\lceil\frac{\sqrt{P}}{2}\right\rceil \times \frac{N^2}{4P}$
Hypercube	$\log_2(P)$	$\left\lceil\frac{N^2}{2P}\right\rceil$
de Bruijn	$\frac{\log_2(P)}{2}$	$\Theta\left(\left\lceil\frac{N^2\log_2(P)}{2P}\left(1-\frac{\sqrt{P}}{P}\right)\right\rceil\right)$

Proof. 1. Transmission rate factor.

• For the grid and torus, we at least must exchange the two outerdiagonal submatrices. The total number of data inside each block is $(N^2/P) \times (P/4)$ and the maximum number of available links is $2 \times (\sqrt{P}/2)$ for the grid [5] and $4 \times (\sqrt{P}/2)$ for the torus.

• For the hypercube, the lower bound is obtained by remarking that the number of links leaving one outerdiagonal submatrix is $2 \times (P/4) = P/2$ for exchanging $(N^2/P) \times (P/4)$ data.

• For the de Bruijn, the result is a direct corollary of the following lemma:

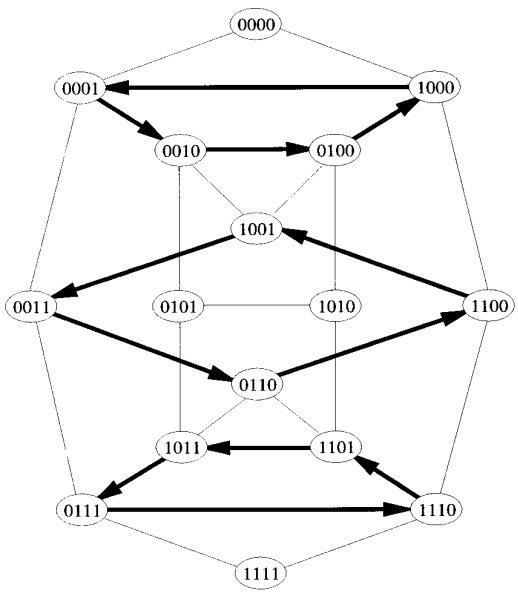
LEMMA 1 [19]. *The bisection of de Bruijn* $(2, n)$ *is equal to* $\Theta(P/\log_2(P))$.

We construct a bipartition of the de Bruijn as follows: we call V the initial set of nodes of the de Bruijn from which we have suppressed symmetric nodes. The construction of the two sets Q and Q^t of the bipartition consists in removing one node q of V and putting it in Q, and removing q^t and putting it in Q^t, until $V = \emptyset$. We add half of the symmetric nodes in Q and the other half in Q^t. It is clear that Q and Q^t are a bipartition of the de Bruijn.

Now we must prove that this bipartition realizes the bisection. We recall that the proof of the de Bruijn bisection is based on its representation in the complex plan [19]. So, we must show that all nodes q in Q belong to the upper half plan, denoted P_1,[5] and that all nodes q^t in Q^t belong to the lower half plane P_2.[6] Thus, if $z \in P_1$ and $z' \in P_2$, then they verify the relation $z = \omega^{n/2} \times z'$.

Consider $q = q_{n-1}\cdots q_0$ a node of the de Bruijn; then its representation in the complex plan is $\overline{q} = \sum_{j=0}^{n-1} q_q \times \omega^j$, where $\omega^j = e^{-2i\pi j/n}$.

[5] $P_1 = \{z = x + iy \in \mathbb{C}, \text{ such that } y > 0\}$.
[6] $P_2 = \{z = x + iy \in \mathbb{C}, \text{ such that } y < 0\}$.



Transposition cycles on the De Bruijn

Generator of the transposition cycles

1000
1100
1110

FIG. 15. Transpose on a de Bruijn (2, 4).

TABLE II
Execution Times for the Different Algorithms Using S&F Routing

Torus	$\frac{\sqrt{P}}{2}\left(\sqrt{\beta}+\sqrt{\frac{N^2}{2P}\tau}\right)^2$
de Bruijn	$\frac{\log_2(P)}{2}\left(\beta+\frac{N^2}{2P}\tau\right)$

LEMMA 2. *Let q be a node of the de Bruijn graph belonging to Q, and q^t its transposed node, where $q^t \in Q^t$. Then $\overline{q^t} = \omega^{n/2} \times \overline{q}$.*

Proof. If $q = q_{n-1}\cdots q_{n/2}q_{(n/2)-1}\cdots q_0$, then $q^t = q_{(n/2)-1}\cdots q_0 q_{n-1}\cdots q_{(n/2)}$ and

$$\begin{aligned}\overline{q^t} &= q_{(n/2)} + q_{(n/2)+1}\omega^1 + \cdots + q_{n-1}\omega^{(n/2)-1} + q_0\omega^{n/2} \\ &\quad + \cdots + q_{(n/2)-1}\omega^{n-1} \\ &= q_0\omega^{n/2} + \cdots + q_{(n/2)-1}\omega^{n-1} + q_{n/2}\omega^n \\ &\quad + q_{(n/2)+1}\omega^{n+1} + \cdots + q_{n-1}\omega^{n+(n/2)-1} \\ &= \omega^{n/2}(q_0 + \cdots + q_{(n/2)-1}\omega^{(n/2)-1} + q_{(n/2)}\omega^{(n/2)} \\ &\quad + q_{(n/2)+1}\omega^{(n/2)+1} + \cdots + q_{n-1}\omega^{n-1}) \\ &= \omega^{n/2} \times \overline{q}.\end{aligned}$$

Thus the maximum number of links which connect Q to Q^t is $\Theta\,(P/\log_2(P))$. The number of nodes in Q which must exchange their N^2/P data with a node in Q^t is exactly equal to $(P/2) - (\sqrt{P}/2)$ and they can use at most $\Theta(P/\log_2(P))$ links. Thus the lower bound for the transmission rate is equal to

$$\Theta\left(\frac{N^2\log_2(P)}{2P}\left(1-\frac{\sqrt{P}}{P}\right)\right).$$

2. The minimum number of steps using S&F routing corresponds to the maximum distance to cross.

• For the grid, torus, and hypercube networks, the number of start-ups is straightforward to obtain since the maximum distance to cross is equal to the diameter.

• For the de Bruijn, the bound for the start-up is obtained by remarking that the maximum distance to cross is $\log_2(P)/2$.

6.2. Optimality Results

From the previous results, we verify in Table II that the S&F version of the algorithm proposed in this paper is optimal for the de Bruijn in terms of start-up and asymptotically optimal for the transmission rate (when the matrix size or the number of processors increases). For the torus, the S&F version of the algorithm reaches asymptotically the start-up lower bound (when the matrix size increases) and is about a factor 2 of the transmission rate lower bound.[7]

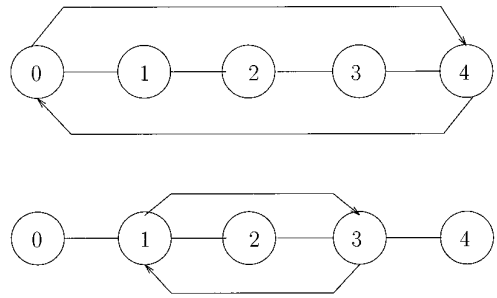


FIG. 16. PT on a path of five processors using wormhole routing.

7. ADAPTATION TO WORMHOLE

The matrix transpose on 2D grids using wormhole routing for block allocations have been already studied by Ho *et al.* [5] and by Tsai and McKinley [28]. These two approaches differ by the fact that the first one aims to minimize the transmission rate factor and the second the number of steps. The basic idea of the first approach is to use the recursive transpose algorithm with pipelined phases. The total number of steps of this algorithm for square grids of P nodes, with $P = 2^k$, is $\mathcal{O}(\sqrt{P}/3)$, and the transmission rate factor is $\mathcal{O}(N^2/3\sqrt{P})$.

Tsai and McKinley use *an extended dominating node approach* in order to minimize the number of steps. They define particular nodes (diagonal nodes) which collect the information to be exchanged and then redistribute them. This recursive algorithm is designed for squared grids of P nodes, with $P = 2^k$. The number of steps is equal to $\log_2\sqrt{P}$ and the factor of the transmission is equal to

$$\left\lfloor\frac{5^{\log_2 P/4}}{2}\right\rfloor \times \frac{N^2}{P}.$$

[7] The multiple paths recursive transpose algorithm proposed in [16] for hypercubes is asymptotically optimal in term of transmission rate.

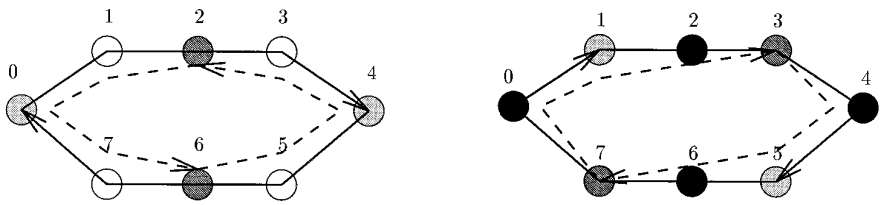


FIG. 17. CT on eight processors using wormhole routing.

TABLE III
Lower Bounds Using Wormhole

	Number of steps	Transmission rate
Grid	$\log_5\left(\frac{\sqrt{P}-1}{2}\right)$	$\left\lceil\frac{\sqrt{P}}{2}\right\rceil \times \frac{N^2}{2P}$
Torus	$\log_5\left(\frac{\sqrt{P}-1}{4}\right)$	$\left\lceil\frac{\sqrt{P}}{2}\right\rceil \times \frac{N^2}{4P}$
Hypercube	$\log_{\log_2(P)+1}\left(2\frac{P-\sqrt{P}}{P}\right)$	$\left\lceil\frac{N^2}{2P}\right\rceil$
de Bruijn	$\Theta\left(\log_5\left(\log_2(P)\times\left(\frac{P-\sqrt{P}}{2P}\right)\right)\right)$	$\Theta\left(\left\lceil\frac{N^2\log_2(P)}{2P}\left(1-\frac{\sqrt{P}}{P}\right)\right\rceil\right)$

The methodology introduced in this paper can be extended to wormhole routing, we just must change the two basic PT and CT algorithms in order to minimize the number of steps. The principle of these algorithms are presented in Figs. 16 and 17. The execution time of these two basic algorithms, taking into account the bidirectional links, are

$$T_{\text{PT-}Worm} = \frac{l}{2}(\beta + L\tau)$$

$$T_{\text{CT-}Worm} = \left\lceil\frac{l+1}{4}\right\rceil (\beta + L\tau).$$

So if we apply these basic schemes to the different topologies, it leads to the following execution times:

$$T^{Grid-Worm} = (\sqrt{P}-1)\left(\beta + \frac{N^2}{P}\tau\right)$$

$$T^{Torus-Worm} = \left(\frac{\sqrt{P}-1}{2}\right)\left(\beta + \frac{N^2}{2P}\tau\right)$$

$$T^{DeBruijn-Worm} = \frac{\log_2(P)}{4}\left(\beta + \frac{N^2}{2P}\tau\right).$$

As for S&F routing, we can establish the lower bounds in terms of both number of steps and transmission rate factors (see Table III). One can note that the lower bounds on the transmission rate factor using wormhole and S&F are equal.

For the number of steps using wormhole routing, we use the following:

LEMMA 3. *Given any graph G of bisection $B(G)$ and maximum degree $\Delta(G)$ with P nodes, the minimum number of steps to achieve a direct transpose is*

$$\left\lceil \log_{(\Delta(G)+1)}\left(\frac{P-\sqrt{P}}{2B(G)}\right)\right\rceil .$$

Proof. We consider the bipartition of the graph G composed by the sets Q and Q^t defined previously for the de Bruijn. For achieving a direct transpose, we must establish $(P-\sqrt{P})/2$ paths between Q and Q^t. Let us consider an arc e which belongs to the set of arcs which are between Q and Q^T. At the first step, the maximum number of paths which can pass through e is equal to $(\Delta(G)+1)^0$. At step t, $(\Delta(G)+1)^{t-1}$ paths have been established using e. Since we have only $B(G)$ arcs between Q and Q^t, it needs at least t steps, such that $(\Delta(G)+1)^{t-1} \times B(G) \geq (P-\sqrt{P})/2$, thus the minimum number of steps is greater than or equal to

$$\left\lceil \log_{(\Delta(G)+1)}\left(\frac{P-\sqrt{P}}{2B(G)}\right)\right\rceil .$$

Thus, we can derive the results given in Table III from the previous lemma and from the values of bisection for the different considered graphs given in Table IV.

Note that the lower bounds for the number of steps are tight since we use the maximum degree of the graph. Indeed, we could take into account the fact that the degree of some nodes in the grid and de Bruijn are less than this maximum degree. One can also notice that all the presented algorithms using wormhole commutation mode are conflict free since we use partitions of the graphs.

TABLE IV
Bisection of the Different Considered Graphs

Graph	Bisection
Grid	$\sqrt{P}$
Torus	$2\sqrt{P}$
Hypercube	$\frac{P}{2}$
de Bruijn	$\Theta\left(\frac{P}{\log_2(P)}\right)$

8. CONCLUSION

We have presented in this paper some matrix transpose algorithms assuming consecutive and cyclic block allocations on the torus and de Bruijn networks. We have first presented a powerful method for elementary transposes on paths and cycles. Then, as the considered networks can be partitioned into paths or cycles, we have applied this analysis for the complete matrix transpose. In both torus and de Bruijn networks, we have derived some improvements by using half of each message on idle links. Moreover, a pipelined strategy makes it possible to decrease the transpose time on the torus. These two algorithms are the best known at this time.

Moreover we have shown how to use the same methodology using wormhole mechanism just by adapting the basic algorithms of exchanging (PT and CT). This leads to conflict-free algorithms for transposing a matrix on the different topologies using wormhole.

APPENDIX

TABLE V
Table of Cycle Generators for $n = 6$

Level	Cycle length	Generator
C_6	1	111111
C_5	6	111110
C_4	6	111100
C_4	6	111010
C_3	6	111000
C_4	3	110110
C_3	6	110100
C_3	6	110010
C_2	6	110000
C_3	2	101010
C_2	6	101000
C_2	3	100100
C_1	6	100000
C_0	1	000000

TABLE VI
Table of Cycles for $n = 6$

111111					
111110	111101	111011	110111	101111	011111
111100	111001	110011	100111	001111	011110
111010	110101	101011	010111	101110	011101
111000	110001	100011	000111	001110	011100
110110	101101	011011			
110100	101001	010011	100110	001101	011010
110010	100101	001011	010110	101100	011001
110000	100001	000011	000110	001100	011000
101010	010101				
101000	010001	100010	000101	001010	010100
100100	001001	010010			
100000	000001	000010	000100	001000	010000
000000					

ACKNOWLEDGMENT

The authors thank C. T. Ho for fruitful discussions.

REFERENCES

1. Bermond, J. C., and Peyrat, C. De Bruijn and Kautz networks: A competitor for the hypercube? In Andre, F., and Verjus, J. P. (Eds.). *Proceedings of the 1st European Workshop on Hypercubes and Distributed Computers, Rennes.* North-Holland, Amsterdam, 1989, pp. 279–293.
2. Boppana, R., and Raghavendra, C. S. Optimal self-routing of linear-complement permutations in hypercubes. *IEEE Trans. Comput.* (1990), 800–808.
3. Choi, J., Dongarra, J., and Walker, D. The design of scalable software libraries for distributed memory concurrent computers. In Dongarra, J., and Tourancheau, B. (Eds.). Environments and Tools for Parallel Scientific Computing. Elsevier, Amsterdam, 1993, pp. 3–15.
4. Choi, J., Dongarra, J., and Walker, D. Parallel matrix transpose algorithms on distributed memory concurrent computers. Tech. Rep. ORNL/TM-12309, Oak Ridge National Laboratory, Oct. 1993.
5. Ding, K. S., Ho, C. T., and Tasy, J. J. Matrix transpose on meshes with wormhole and XY routing. *Sixth IEEE Symposium on Parallel and Distributed Processing.* Dallas, TX, Oct. 1994, pp. 656–663.
6. Edelman, A. Optimal matrix transposition and bit reversal on Hypercubes: All-to-all personnalized communication. Tech. rep., Thinking Machines Corporation, 1989.
7. Eklundh, J. O. A fast computer method for matrix transposing. *IEEE Trans. Comput.* **21** (1972), 801–803.
8. Flanders, P. M. A unified approach to a class of data movements on an array processor. *IEEE Trans. Comput.* **31** (1982), 809–819.
9. Forum, H. P. F. High Performance Fortran Language Specification (version 1.0), 1993.
10. Fredricksen, H. A new look at the de Bruijn graph, Discrete *Appl. Math.* **37/38** (1992), 193–203.
11. Graham, R. L., Knuth, D. E., and Patashnik, O. *Concrete Mathematics—A Foundation for Computer Science.* Addision Wesley, Reading, MA, 1990.
12. Gupta, A., and Kumar, V. *The Scalability of FFT on Parallel Computers. IEEE Trans. Parallel Distrib. Systems* **4** (1993), 922–932.
13. Hendrickson, B., Leland, R., and Plimpton, S. An efficient parallel algorithm for matrix-vector multiplication. Tech. rep., Sandia National Laboratories, Albuquerque, NM 87185, 1994. Published in *Internat. J. High Speed Comput.*
14. De Rumeur, J. Communications dans les Réseaux de Processeurs, Masson, 1995.
15. Johnsson, S. L. Communication efficient basic linear algebra computations on hypercube architectures. *J. Parallel Distrib. Comput.* **4** (1987), 133–172.
16. Johnsson, S. L., and Ho, C. T. Algorithms for matrix transposition on Boolean *N*-cube configured architectures. *SIAM J. Matrix Anal. Appl.* **9** (1988), 419–454.
17. Johnsson, S. L., and Ho, C. T. Optimum broadcasting and personnalized communications in hypercubes. *IEEE Trans. Comput.* **38** (1989), 1249–1267.
18. Johnsson, S. L., and Ho, C. T. Shuffle permutations on Boolean cubes. Tech. rep., Yale University Department of Computer Science, Oct. 1988.
19. Leighton, F. T. *Introduction to Parallel Algorithms and Parallel Architectures: Arrays–Trees–Hypercubes.* Morgan Kaufman, San Mateo, CA, 1992.
20. Lewis, J., and van de Geijn, R. Distributed memory matrix-vector multiplication and conjugate gradient algorithms. *Proceedings of the Supercomputing Conference.* 1993.

21. MacMahon, P. A. Application of a theory of permutations in circular procession to the theory of numbers. *Proceedings of the London Mathematical Society.* 1892, pp. 305–313.
22. McKinley, P., Tsai, Y., and Robinson, D. A survey of collective communication in wormhole-routed massively parallel computers. Tech. Rep. MSU-CPS-94-35, Michigan State University, June 1994.
23. Miller, R., and Tanimoto, S. L. Detecting repeated patterns on mesh computers. Tech. rep., University of Buffalo, 1990.
24. Nassimi, D., and Sahni, S. Data broadcasting in SIMD computers. *IEEE Trans. Comput.* **30** (1981), 101–107.
25. Ni, L., and McKinley, P. A survey of routing techniques in wormhole networks. *Computer* (1993), 62–76.
26. Saad, Y., and Schultz, M. H. Topological properties of hypercubes. *IEEE Trans. Comput.* **37** (1988), 867–871.
27. Saad, Y., and Schultz, M. H. Data communication in parallel architectures. *Parallel Comput.* **11** (1989), 131–150.
28. Tsai, Y., and McKinley, P. An extended dominating node approach to collective communication in all-port wormhole-routed 2D meshes. *Proceedings of the SHPC Conference.* IEEE, May 1994, pp. 199–206.

CHRISTOPHE CALVIN was born in Saint Raphaël, France, in 1967. He has received a Ph.D. from the Institut National Polytechnique de Grenoble in Applied Mathematics (1995). He now holds a post-doctorate position at CEA (Commissariat à l'Énergie Atomique) in Grenoble. He is involved in the design and the implementation of a code which is used as a platform for the development of parallel thermohydraulic applications based on decomposition domain methods. His research activities concern the improvement of the efficiency of parallel applications: optimization of communications, design of efficient parallel numerical algorithms, study of communication overlapping in parallel algorithms, and implementation of real applications on parallel machines.

DENIS TRYSTRAM was born in Paris in 1958. He has received two Ph.D.'s from the Institut National Polytechnique de Grenoble in respectively Applied Mathematics (1984) and Computer Science (1988). He has been professor in computer science at the Institut National Polytechnique de Grenoble since 1991. Since 1990, he has been the head of the "Parallel Processing" group at LMC-IMAG. He manages an interdisciplinary regional group for parallelizing practical applications (ParAppli), and he is currently regional editor for the *Parallel Computing Journal.* His research activities concern all aspects of the study of the impact of parallelism on the conception of efficient algorithms (namely, model and design of parallel algorithms, scheduling and mapping, optimization of communications, and implementation of real applications on parallel machines). He has published five books, and about 50 articles in international journals and as many international conferences.

Received October 5, 1993; revised February 9, 1995; accepted September 24, 1995